A Rule-Based Prolog Expert System for Course Timetabling

Qiang Xue - 40300671, Aamna Fawad - 40329487, Avneet Kaur - 40326551

Course: *COMP 6591* Date: *15 February 2026*

Abstract

Course timetabling is a combinatorial optimization problem with hard feasibility constraints (e.g., no conflicts) and soft quality constraints (e.g., compact curricula, minimum working days) [2]. This project proposes a rule-based expert system implemented in SWI-Prolog to solve the ITC2007 Track 2 Curriculum-Based Course Timetabling benchmark [1]. We will model the domain as a knowledge base of declarative constraints and build an inference/solving engine that constructs feasible timetables using a construction heuristic with backtracking and randomized restarts. The system will be evaluated on multiple ITC2007 instances using feasibility rate, penalty score, and runtime.

1 Problem Statement

We aim to build an expert system that generates weekly course timetables from ITC2007 Track 2 (Curriculum-Based Course Timetabling) instances [1, 3]. Each instance defines:

- a set of courses (teacher, number of lectures, minimum working days, number of students),
- a set of rooms (capacity),
- curricula (groups of courses that share students),
- and unavailability constraints (course cannot be scheduled in a specific time slot).

The goal is to assign every lecture to a *day*, *period*, and *room* satisfying all hard constraints and minimizing a soft-constraint penalty.

2 Objectives and Success Criteria

2.1 Objectives

1. Parse ITC2007 `.ctt` data files and build an internal model.
2. Encode hard constraints as Prolog rules and validate feasibility.
3. Encode soft constraints and compute the objective (penalty).
4. Implement a solver that produces feasible solutions.
5. Provide reproducible experiments (scripts + CSV summaries) and write-up.

2.2 Success Criteria

- **Correctness:** solutions pass a hard-constraint validator.
- **Coverage:** run on all provided instances (e.g., 21 competition instances).
- **Quality:** penalty is reported and shows improvement over a simple baseline (when feasible).
- **Reproducibility:** results re-generated with a single command producing a summary CSV.

3 Dataset

Instances are obtained from the official ITC2007 website:

https://www.eeecs.qub.ac.uk/itc2007/Login/SecretPage.php

4 Constraints (Domain Knowledge Base)

4.1 Hard Constraints

Hard constraints must always hold:

- **Lecture completeness:** all required lectures for each course are scheduled.
- **Room conflict:** at most one lecture per room per time slot.
- **Teacher conflict:** a teacher teaches at most one lecture per time slot.
- **Curriculum conflict:** courses in the same curriculum do not overlap.
- **Unavailability:** forbidden (course, day, period) slots are never used.

4.2 Soft Constraints (Penalty)

Soft constraints define solution quality (lower is better). We will implement the Track 2 penalties:

- **Room capacity:** penalty when students exceed room capacity.
- **Minimum working days:** penalty when a course is spread across fewer than required days.
- **Curriculum compactness:** penalty for isolated lectures in a curriculum.
- **Room stability:** penalty when a course uses multiple rooms.

5 Methodology and System Design

5.1 Expert-System Architecture

We separate *knowledge* (constraints) from *inference* (solving) [4]:

- **Parser/Model:** reads `.ctt` into structured facts.
- **Rule base:** Prolog predicates representing constraints.
- **Solver engine:** constructs a schedule using guided search.
- **Validator/Scorer:** checks hard constraints and computes penalty.
- **Output:** writes `.sol` and CSV summaries.

5.2 Solving Strategy (Construction-Only)

Given the Phase 1 time constraints, we will focus on a construction-only solver that aims to reliably produce feasible timetables and report their penalties.

1. **Lecture ordering:** schedule the most constrained lectures first (e.g., high degree in conflicts, many unavailable slots).
2. **Feasible placement:** choose a (day, period, room) assignment that satisfies all hard constraints, preferring placements that reduce immediate soft penalties (e.g., adequate room capacity).
3. **Backtracking + randomized restarts:** if the solver gets stuck, it backtracks and/or restarts with different tie-breaking choices to improve feasibility rates.

We will incorporate per-instance time limits and record runtime.

6 Evaluation Plan

6.1 Metrics

- Feasibility rate (% of instances producing valid schedules).
- Total penalty (objective value) on feasible instances.
- Runtime per instance (seconds).

6.2 Experimental Procedure

- Run a simple baseline (e.g., first-fit greedy without restarts) to establish a reference.
- Run the construction solver (ordering + penalty-aware placement + restarts) under identical time limits.
- Aggregate results into a CSV summary and include tables in the final report.

7 Timeline

Week	Milestones
1	Parse `.ctt`; instance validator; mini end-to-end run
2	Implement hard constraints; solution validator; CI tests
3	Implement soft constraints; penalty computation; baseline benchmarks
4	Construction solver refinements; full benchmark runs; result tables
5	Write report/paper; prepare presentation and demo

8 Work Breakdown Structure (WBS) and Contributions

Table 1 summarizes the planned work packages and expected contributions. Tasks may be adjusted as implementation findings emerge, but ownership ensures accountability and parallel progress.

WBS	Work Package	Owner(s)	Outputs
1.0	Project setup, repo structure, CI, coding conventions	Qiang, Aamna	Working repo + CI
1.1	ITC2007 Track 2 parser + instance model + sanity checks	Qiang	Parser + unit tests
2.0	Hard-constraint rule base (teacher/room/curriculum/unavailability/completeness)	Aamna	Validator + tests
2.1	Soft-constraint scoring (capacity, min working days, compactness, stability)	Avneet	Penalty module + tests
3.0	Construction heuristic (greedy + restarts) + feasibility diagnostics	Qiang	Baseline solver
3.1	Penalty-aware construction (ordering + room/day heuristics)	Avneet, Aamna	Construction solver (quality-focused)
4.0	Benchmarking harness (batch runs, CSV summary, reproducibility)	Qiang	summary.csv
5.0	Proposal + report + paper drafts (writing + figures/tables)	All	LaTex Documents
6.0	Presentation slides + demo script + rehearsal	All	Slides + demo

Table 1: Work Breakdown Structure and contribution plan.

9 Deliverables

- Source code (Prolog), tests, and scripts for reproducible runs.
- Proposal (this document), final report, short research paper, and presentation slides.

10 Related Work (Starter)

Curriculum-based course timetabling has been extensively studied in the literature [2]. The ITC2007 competition [1] established standardized benchmarks and evaluation frameworks for this problem. Various approaches have been explored, including constructive heuristics and broader metaheuristic strategies [5]. Recent work has focused on algorithm selection and instance space analysis to understand problem difficulty [3]. Rule-based expert system approaches have also been applied to university scheduling problems [4], which motivates our Prolog-based approach.

References

[1] International timetabling competition 2007 (itc2007). https://www.eeecs.qub.ac.uk/itc2007/. Accessed: 2026-02-04.

[2] Andrea Bettinelli, Valentina Cacchiani, Roberto Roberti, and Paolo Toth. An overview of curriculum-based course timetabling. *Top*, 23(2):313–349, 2015.

[3] Arnaud De Coster, Nysret Musliu, Andrea Schaerf, Johannes Schoisswohl, and Kate Smith-Miles. Algorithm selection and instance space analysis for curriculum-based course timetabling. *Journal of Scheduling*, 25(1):35–58, 2022.

[4] L Guyette, K Hamidian, and JO Tuazon. A rule-based expert system approach to class scheduling. *Computers & electrical engineering*, 20(2):151–162, 1994.

[5] Rong Qu, Edmund K Burke, Barry McCollum, Liam TG Merlot, and Sau Y Lee. A survey of search methodologies and automated system development for examination timetabling. *Journal of scheduling*, 12(1):55–89, 2009.